

Student Name: Sultangazyyeva Altynay

Student ID: 230103346

Hardware / CPU: Apple M2 (8 cores: 4 performance + 4 efficiency), MacBook Air

Date: 01.10.2026 ; 06-p

Benchmark	Threads	Size	Time (s)	Speedup	Efficiency (%)
Monte Carlo	1 (base)	120,000,000	1.002	1.00x	100.0
Monte Carlo	2	120,000,000	0.512	1.95x	97.7
Monte Carlo	4	120,000,000	0.268	3.74x	93.4
Monte Carlo	Max (8)	120,000,000	0.226	4.42x	55.3
Mandelbrot (Rows)	8	2500 x 2500	1.561	N/A	N/A
Mandelbrot (Cols)	8	2500 x 2500	1.526	N/A	N/A
Heat Stencil (float64)	8	1500 x 1500 x 300	0.152	N/A	N/A

Challenge-1:

A. $T_1 = 1.002$ s, $T_{\max}(8 \text{ threads}) = 0.226$ s.

B. Numba compiles the function with JIT on its first call. Without the warm-up, the measured time would include LLVM compilation and thread-pool startup, which would inflate T_1 and distort the speedup. The warm-up compiles the code first, so only machine-code execution is timed.

C. No. Efficiency was 97.7% at 2 threads and 93.4% at 4 threads, but dropped to 55.3% at 8 threads (speedup only 4.42x). The M2 has 4 performance and 4 efficiency cores. Threads 5–8 land on slower E-cores, and since work is split equally, all threads wait for the slowest one. Other factors: Amdahl's Law (serial thread startup and final reduction), and frequency reduction on the fanless MacBook Air. Memory bus contention is negligible here because Monte Carlo barely touches memory. Hyperthreading does not apply, since Apple Silicon has no SMT.

D. Numba recognizes `inside_circle += 1` in a prange as a reduction. Each thread gets a private copy initialized to 0, increments it independently (no data race), and the private copies are summed after the loop. This is equivalent to OpenMP `reduction(+:inside_circle)`.

Challenge-2:

A. Row-parallel: 1.561 s, column-parallel: 1.526 s. The column decomposition was about 2% faster, which is practically a tie. The task is compute-bound: the while iteration loop dominates, while memory writes (6.25M values) are negligible, so the memory-access pattern barely affects total time.

B. NumPy arrays are C-contiguous, so elements of a row are adjacent in memory. In `img[r, c]` with the inner loop over `c` (row-parallel), writes are sequential and fill each cache line completely, and each thread writes its own rows. With the inner loop over `r` (column-parallel), writes jump by a stride of $W \times 4$ bytes, touching a new cache line every time. Neighboring threads write to adjacent

columns, which share cache lines, causing false sharing and coherence invalidations. Here this effect is hidden by the heavy computation.

C. The top 20% of rows lie mostly outside the Mandelbrot set, so points escape in 2–3 iterations and that thread finishes almost immediately. The center 20% of rows cross the set, where points run up to `max_iter = 1000` iterations. With static scheduling, the cheap thread idles at the barrier while the other struggles, and total time equals the slowest thread. OpenMP dynamic scheduling splits the loop into small chunks held in a shared queue; a thread that finishes a chunk grabs the next one, so expensive chunks are spread across all threads and idle time nearly disappears.

D. Attached: `mandelbrot_output.png`.

Challenge-3:

A. Runtime = 0.152 s, throughput = 4455.30 Megacells/sec (float64).

B. With float32: 0.076 s and 8833.51 Megacells/sec, so runtime decreased by a factor of 1.98x (about 2x). A stencil has very low arithmetic intensity (a few flops per many bytes loaded), so it is limited by memory traffic, not compute. Halving the data type halves the bytes moved through the memory bus and lets twice as many values fit in cache and in each cache line, while the amount of computation stays the same.

C. According to the Roofline model, performance is limited by $\min(\text{peak compute}, \text{memory bandwidth} \times \text{arithmetic intensity})$. The stencil's low arithmetic intensity places it under the memory-bandwidth roof. Memory bandwidth is a shared resource of the whole chip: once a few cores saturate the memory bus, additional cores only wait for data from RAM. Doubling the cores raises peak compute but not bandwidth, so speedup is well below 2x.

```
Hardware Threads Detected: 8

Threads      | Time (s)      | Speedup      | Efficiency (%)
-----|-----|-----|-----|
1            | 1.0016        | 1.00         | 100.0
2            | 0.5124        | 1.95         | 97.7
4            | 0.2681        | 3.74         | 93.4
8            | 0.2264        | 4.42         | 55.3

Row-Parallel Render Time: 1.561 s
Column-Parallel Render Time: 1.526 s
Saved image: mandelbrot_output.png

Heat float64: 0.152 s, 4455.30 Megacells/s
Heat float32: 0.076 s, 8833.51 Megacells/s (runtime ratio f64/f32 = 1.98x)

=== TABLE 1 ===
| Challenge | Threads | Size | Time (s) | Speedup | Efficiency (%) |
|---|---|---|---|---|---|
| Monte Carlo | 1 thr | 120M | 1.002 | 1.00x | 100.0 |
| Monte Carlo | 2 thr | 120M | 0.512 | 1.95x | 97.7 |
| Monte Carlo | 4 thr | 120M | 0.268 | 3.74x | 93.4 |
| Monte Carlo | 8 thr | 120M | 0.226 | 4.42x | 55.3 |
| Mandelbrot (Rows) | 8 thr | 2500x2500 | 1.561 | N/A | N/A |
| Mandelbrot (Cols) | 8 thr | 2500x2500 | 1.526 | N/A | N/A |
| Heat Stencil f64 | 8 thr | 1500x1500x300 | 0.152 | N/A | N/A |
| Heat Stencil f32 | 8 thr | 1500x1500x300 | 0.076 | N/A | N/A |
(venv) altxw@MacBook-Air-Altynay Desktop %
```

Mandelbrot 2500x2500 (Render: 1.56s)

